CAGE: When Does Communication-Graph Structure Pay Off for Rank Placement?

Divyansh Verma, Aniruddha Badhani, Ashutosh Singh
Dept. of Computer Science, Noida Institute of Engineering and Technology, Greater Noida, India
{0241cs202, 0241cs161, 0241cs203}@niet.co.in

***Abstract*—Placing communicating processes close together is a decades-old idea, but whether the communication graphs of modern distributed training jobs hold structure that a placement can exploit, beyond what the launcher's rank order already captures, has rarely been tested with statistical care. We present CAGE, a pipeline that turns communication traces into traffic graphs, expands collectives by the algorithm that ran, tests community structure against a degree-preserving null model, and recommends exact-capacity placements. Placements are scored under two regimes: collectives held in rank order (MPI-like) and collectives re-ringed for each placement (NCCL-like). On models of data-parallel, Megatron 3D-parallel, mixture-of-experts (MoE) and stencil workloads, three results stand out. First, raw modularity misleads: a 64-rank ring scores $Q = 0.74$ with $z = -1.1$ against its null model. Second, against block placement, graph-aware placement gains nothing for data-parallel and default-order Megatron jobs. It gains only where rank order disagrees with structure: -93% cross-host bytes for scrambled Megatron ranks, -43% and -25% for 2D and 3D stencils, and -56% for MoE with routing affinity 0.6, falling to zero without affinity. Third, a bytes-only graph cannot find a slow NIC (hit rate 0.12, chance 0.125), while a time-weighted graph does (0.75–1.00). All numbers are scenario values from synthetic workloads and an analytic bandwidth model, not hardware measurements; we state what real traces must show to confirm them.**

***Index Terms*—communication graphs, process placement, graph partitioning, modularity, null models, distributed training, NCCL, MPI**

I. Introduction

Distributed training jobs spend a large share of each step moving data between accelerators. When two heavily communicating ranks sit on different hosts, their traffic crosses the network interface instead of the much faster intra-host links. The idea that a scheduler should place processes using their communication pattern goes back to Bokhari's mapping problem [1] and is the basis of tools such as MPIPP [2], TreeMatch [3], Scotch [4] and generic topology mapping [5]; MPI even standardises rank reordering from a declared graph [6], [7]. We therefore do *not* claim that building a communication graph and partitioning it is new.

What is missing is an honest answer to a narrower question: *for the communication patterns of modern training jobs, when does graph-aware placement beat the placement a launcher already produces?* Three pitfalls make the obvious experiment misleading. (i) Modularity is high for graphs with no community structure at all, such as rings and sparse random graphs [8], so a fixed threshold such as "$Q > 0.3$" reports structure that is not there. (ii) Round-robin placement is a strawman; the realistic default is *block* placement (SLURM block, `torchrun`), which already co-locates consecutive ranks. (iii) Libraries such as NCCL rebuild ring order from the detected topology, so traffic of a collective expanded in the original rank order is not fixed when ranks move.

CAGE is built around these pitfalls. Our contributions are:

1) A trace-to-placement pipeline that expands collectives by the algorithm that ran, keeps per-step and per-operation detail, and scores placements in two regimes: collectives fixed in rank order, and collectives re-ringed for each candidate placement (Section III).
2) A null-model protocol for community structure (degree-preserving rewiring, ≥ 19 samples, $z > 3$, $p \leq 0.05$) and exact-capacity partitioning that is never worse than block placement by construction.
3) An empirical characterisation across data-parallel, Megatron 3D-parallel, MoE and stencil workloads that includes negative results, a simple decision rule for when placement pays, and evidence that bytes-only graphs cannot locate time bottlenecks (Section V).

Scope of evidence. Every workload in this paper is a synthetic model with known ground truth, and every time is the output of an analytic bandwidth model. We call these numbers *scenario values*. They rank placements; they do not predict wall-clock speedups. Section VI lists what real traces must confirm.

II. Background and Related Work

Topology and process mapping. Mapping a task graph onto a machine graph is NP-hard in general [1]. Practical tools extract a communication matrix and partition it: MPIPP profiles MPI programs and places processes on SMP clusters [2]; TreeMatch maps onto hierarchical machines [3]; Scotch performs dual recursive bipartitioning of process and architecture graphs [4]; Hoefler and Snir compare generic mapping heuristics [5]. A survey is given in [9]. CAGE uses the same core machinery (METIS [10] and Kernighan–Lin-style refinement [11]). Our contribution is the measurement protocol and the characterisation, not the partitioner.

Communication analysis tools. Score-P and Vampir show communication matrices [12]; Chakra standardises execution traces for ML workloads [13]; ScalAna applies graph analysis to locate scaling loss [14]. These tools visualise or diagnose; they do not ask when a placement change would help relative to the launcher default.

ML communication. Data parallelism uses ring or hierarchical allreduce [15], [16], [17]; Megatron-LM combines tensor, pipeline and data parallelism with a fixed rank order [18], [19]; MoE models add all-to-all token routing [20], [21], and FasterMoE exploits topology in expert placement [22]. ByteScheduler [23] schedules communication and TopoOpt [24] co-designs topology with parallelisation. CAGE is complementary: it asks whether the traffic graph itself says that ranks are in the wrong place.

Community structure and null models. Modularity [25] optimised by Louvain [26] finds communities but is biased: random and regular graphs reach high Q [8], and modularity has a resolution limit [27]. Degree-preserving rewiring [28] gives the standard null model; we use it for every reported Q.

III. CAGE Design

A. Trace model

A trace is an event table, not a graph. Each event records source rank, destination rank, bytes, message count, step index, operation name and, when the source provides it, duration. The static traffic matrix M (M_{ij} = bytes from i to j), per-step matrices and arbitrary windows are views of the same table. The undirected view is $W = M + M^\top$. Keeping steps lets the same data serve a static analysis (this paper) and iteration-level analysis (future work).

B. Collective expansion

A collective is expanded into the point-to-point transfers of the algorithm that ran, because that is what the network carries. For a ring allreduce of S bytes over p ranks each ring edge carries $2(p-1)S/p$; for ring allgather and reduce-scatter, $(p-1)S/p$; chain broadcast and reduce carry S per edge; all-to-all carries the routed bytes directly [16], [17]. Hierarchical variants (leader-based [15] and rail-based) are compositions of these. The operation name is kept on every event, so a different expansion can replace the assumed one without re-collecting traces. Each recorded library collective is also stored with its member list, so it can be re-expanded for a new placement.

C. Load and hotspot metrics

The strength (weighted degree) of rank i is $s_i = \sum_j W_{ij}$, so $\sum_i s_i = 2\sum_{ij} M_{ij}$: each byte has a sender and a receiver. A unit test enforces this identity. We report load concentration as the Gini coefficient of s and the share of total strength held by the busiest 20% of ranks. Hotspots are ranks whose robust z-score (median and scaled MAD) exceeds 3.5; the robust score avoids masking by a few large hubs.

D. Community structure with a null model

We run Louvain on W and compare its modularity Q with N null graphs drawn by degree-preserving double-edge swaps that carry edge weights [28] (weight shuffling for graphs denser than 80%). We report

$$z = \frac{Q - \overline{Q}_{\text{null}}}{\sigma_{\text{null}}}, \qquad p = \frac{1 + \#\{Q_{\text{null}} \geq Q\}}{1 + N}, \tag{1}$$

and call structure *significant* only if $N \geq 19$, $z > 3$ and $p \leq 0.05$. With $N = 20$ the smallest attainable p is $1/21 \approx 0.048$.

E. Exact-capacity placement

Hosts have a fixed number of accelerator slots g (8 here), so a placement must put exactly g ranks on each host. METIS only bounds imbalance, so we run it with the tightest imbalance factor, repair overfull parts greedily (moving the rank that loses the least cut), and then apply best-improvement pairwise swaps whose gain for $i \in a$, $j \in b$ is

$$\Delta_{ij} = D_{ib} - D_{ia} + D_{ja} - D_{jb} - 2W_{ij}, \tag{2}$$

where D_{ip} is the weight from i to part p. Swaps keep capacity exact. The recommended placement is the lowest-cut result of three seeds (METIS, block, random), so it is never worse than block placement. Parts are finally renamed by Hungarian matching [29] so as few ranks as possible move.

F. Two placement regimes

Fixed. All traffic, with collectives in their recorded rank order, moves with the ranks. This is valid for MPI codes whose collective algorithms follow rank order, and for traffic pinned to rank identity: point-to-point, pipeline sends, halo exchange and expert all-to-all.

NCCL. Rank-pinned traffic moves with the ranks, while every recorded library collective is re-expanded as a ring ordered by host for each candidate placement, as NCCL derives rings from the detected topology. The partitioning objective in this regime adds a clique expansion of each collective (weight $8e/p^2$ per member pair for e ring bytes), which makes an even two-way split cost the same as the two crossing ring edges it causes. Candidates from both objectives and block placement are scored under the NCCL model and the best is kept. For GPU jobs this regime is the headline; the fixed regime overstates what moving ranks can achieve.

G. Cost model

For host h with egress E_h and ingress I_h cross-host bytes, intra-host bytes X_h and m_h messages,

$$T_h = \frac{\max(E_h, I_h)}{B_{\text{nic}} f_h} + \frac{X_h}{B_{\text{intra}}} + \alpha\, m_h, \qquad T = \max_h T_h, \tag{3}$$

with $B_{\text{nic}} = 50$ GB/s per direction (400 Gb/s), $B_{\text{intra}} = 300$ GB/s, $\alpha = 5\,\mu$s and $f_h = 1$ unless a fault is injected. The model ignores compute overlap, dependencies and in-fabric congestion. It ranks placements; it does not predict seconds.

H. Extraction

CAGE reads its own event CSV, NCCL debug logs (`NCCL_DEBUG=INFO`, `SUBSYS=INIT,COLL`), Chakra execution traces in JSON form [13], and the output of a PMPI interposition library that records collectives as collectives, with communicator membership and durations. PMPI cannot see PyTorch GPU jobs, which call NCCL, not MPI. The parsers are tested against hand-written fixtures only; validating them on real logs is future work (Section VI).

TABLE I
SYNTHETIC WORKLOADS (ALL BF16, 8 GPUS PER HOST).

Workload	Model
DP ring	flat ring allreduce of 2 GB gradients per step
DP hier. leader / rail	Horovod-style leader allreduce / rail-optimised allreduce
Megatron TP·PP·DP	8·4·2, L=32, h=8192, seq 4096, 8 microbatches; TP ring allreduce, PP sends (scatter-gather), DP allreduce of the parameter shard
MoE	EP = 64 ranks, 2 experts/rank, top-2, Zipf expert popularity (default 1.0), affinity a, plus dense 2 GB ring allreduce
Stencil	halo exchange of 4 MB per face on 8×8 or 4×4×4×4 grids
SBM	planted partition [30], 64 ranks in 8 blocks, log-normal weights

IV. METHODOLOGY

Workloads. Table I lists the synthetic generators. Rank order follows each launcher's default (block for data parallel, Megatron's TP-fastest order, MPI Cartesian row-major order), so the block baseline is realistic. Two perturbations model a mismatch between rank order and structure: a random relabelling of Megatron ranks ("scrambled"), and MoE data groups interleaved across ranks ($g(r) = r \bmod 8$) with a routing-affinity knob a, the fraction of a token's routing probability placed on experts hosted inside its own data group. Whether real MoE training shows rank-level affinity is an open empirical question; a lets us show both sides of it.

Baselines. Block placement (ranks $[0, g)$ on host 0, ...) is the reference. Round-robin (r mod H) is shown only beside it. Reductions are $1 - x_{\text{CAGE}}/x_{\text{block}}$.

Protocol. Seeds are fixed. Each experiment script states its expected result before it is run, and misses are reported. Every number in this paper is read from a results file generated by the published scripts (Section VII).

V. RESULTS

A. Raw modularity is not evidence (E2)

Table II compares observed Louvain Q with the null-model mean for graphs whose structure is known. A 64-rank ring has $Q = 0.74$ but its null graphs reach 0.76 ($z = -1.1$); a sparse random graph of mean degree 4 has $Q = 0.48$ ($z = 2.1$, $p = 0.095$); Horovod-style hierarchical allreduce has $Q = 0.65$ ($z = -0.8$). All three pass the old "$Q > 0.3$" rule and none has significant structure. The 2D stencil, the planted partition and Megatron are significant ($z = 6.2$, 17.4 and 10.2). MoE without affinity is not ($z = -0.1$).

B. Recovery of planted structure (E1)

Planted partitions with mixing μ (the expected share of a rank's traffic that leaves its block) are relabelled at random, so rank order carries no hint. Fig. 1 shows the mean over five seeds. Louvain NMI [31] falls from 0.99 at $\mu = 0.05$ to 0.51 at $\mu = 0.6$ and 0.27 at $\mu = 0.75$; exact-capacity partition ARI [32] falls from 0.99 to 0.33 and 0.12. Significance tracks recovery: z falls from 25.1 to 3.3 at $\mu = 0.6$ and 0.6 at $\mu = 0.75$, so CAGE's detection limit lies between $\mu = 0.6$ and 0.75. Above it, CAGE reports *no structure* rather than inventing communities.

TABLE II
OBSERVED VS. NULL-MODEL MODULARITY, 20 NULL SAMPLES (E2). SCENARIO VALUES.

Graph	Q	null Q	z	p	Q>0.3	sig.
ring (64)	0.740	0.756	−1.1	1.000	yes	no
random sparse, deg 4	0.482	0.445	2.1	0.095	yes	no
random dense, deg 32	0.080	0.089	−1.9	0.952	no	no
complete, uniform	0.000	0.000	0.0	1.000	no	no
hier. leader (32)	0.653	0.677	−0.8	0.714	yes	no
MoE, no affinity	0.224	0.225	−0.1	0.571	no	no
2D stencil 8×8	0.594	0.500	6.2	0.048	yes	yes
SBM, $\mu \approx 0.15$	0.685	0.445	17.4	0.048	yes	yes
Megatron 8·4·2	0.821	0.661	10.2	0.048	yes	yes

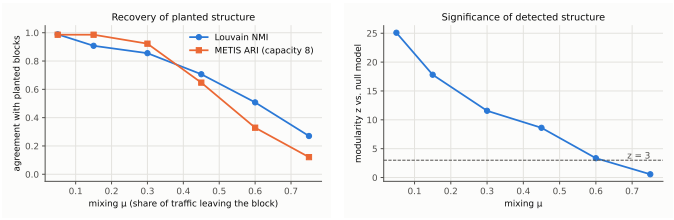


Fig. 1. E1: recovery of planted blocks (left) and null-model z (right) against mixing μ. Scenario values.

C. Workload characterisation (E3)

Table III characterises each workload. The original hypothesis that the busiest 20% of ranks carry 70–80% of traffic fails everywhere: the largest top-20% share is 0.42 (MoE with Zipf 1.5), and collective-based data parallelism is balanced by design (Gini 0.00 for the flat ring and rail allreduce, 0.10 for leader-based allreduce, whose leaders carry the inter-host ring). Skewed MoE routing creates hotspots (6 and 11 ranks above robust $z = 3.5$ for Zipf 1.0 and 1.5) but no community structure; its Q is *below* the null mean at Zipf 1.5 ($z = -5.0$) because a few hub experts connect everything. Megatron and the stencils are significant. Rail-optimised allreduce is significant ($z = 5.7$) while leader-based allreduce is not ($z = 0.0$), yet in both cases the structure is defined by the algorithm's own host layout: significance alone does not mean a placement will help, and Section V-D shows that, as we test it, it is not required either.

D. When placement beats block (E4)

Table IV is the headline. In the NCCL regime CAGE gains nothing for DP ring, hierarchical DP and Megatron in its default order, and moves no ranks: block placement is already optimal, and CAGE says so. Megatron is strongly structured ($z = 10.2$) but its rank order already matches that structure. Gains appear only where rank order disagrees with structure: scrambled Megatron ranks (−92.7% cross-host bytes, −78.7% modelled time, 48 ranks moved), the 2D and 3D stencils (−42.9% and −25.0% bytes, because Cartesian row-major order cuts the grid into thin slabs), and MoE with routing affinity 0.6 (−56.3%

TABLE III
LOAD CONCENTRATION AND STRUCTURE, 64 RANKS, 20 NULL SAMPLES (E3). SCENARIO VALUES.

Workload	Gini	top-20%	hotspots	Q	z	p
DP ring	0.00	0.20	0	0.74	−1.1	1.000
DP hier. leader	0.10	0.25	0	0.75	0.0	0.381
DP hier. rail	0.00	0.20	0	0.76	5.7	0.048
Megatron 8·4·2	0.00	0.20	0	0.82	10.2	0.048
MoE Zipf 0	0.01	0.21	0	0.23	2.2	0.048
MoE Zipf 1.0	0.19	0.35	6	0.22	−0.1	0.571
MoE Zipf 1.5	0.27	0.42	11	0.22	−5.0	1.000
MoE Zipf 1, a=0.6	0.15	0.32	6	0.28	2.5	0.048
Stencil 2D 8×8	0.09	0.23	0	0.59	6.2	0.048
Stencil 3D 4×4×4	0.10	0.25	0	0.54	6.8	0.048

TABLE IV
PLACEMENT VS. BLOCK, 64 RANKS (E4). Δ = CHANGE VS. BLOCK; NEGATIVE IS BETTER. SCENARIO VALUES.

	NCCL regime, CAGE			RR	Fixed
Workload	Δbytes	Δtime	moved	Δbytes	Δbytes
DP ring	0.0%	0.0%	0	0.0%	0.0%
DP hier. leader†	0.0%	0.0%	0	+700%	0.0%
Megatron 8·4·2	0.0%	0.0%	0	+1657%	0.0%
Megatron, scrambled	−92.7%	−78.7%	48	−11.1%	−93.9%
Stencil 2D 8×8	−42.9%	−27.3%	40	0.0%	−42.9%
Stencil 3D 4×4×4	−25.0%	−31.4%	32	+50.0%	−25.0%
MoE, $a = 0$	−0.1%	−14.9%	46	0.0%	0.0%
MoE, $a = 0.6$	−56.3%	−48.8%	56	−56.3%	−13.3%

†The hierarchical generator records no re-ringable collectives, so its NCCL row equals the fixed regime. RR = round-robin.

bytes). Round-robin is catastrophic wherever rank order is right (+700% bytes for hierarchical DP, +1657% for Megatron); the flat DP ring is the exception only because NCCL re-rings it to the same cost. It ties CAGE on affinity MoE only because the interleaved data groups (r mod 8) coincide exactly with round-robin hosts.

For MoE without affinity, cross-host bytes do not change (−0.1%) while modelled time falls by 14.9%: the partition shifts load between NICs rather than removing traffic. Since the cost model takes the busiest host, we treat this as a load-balancing side effect, not a structure-driven gain.

Fixed vs. NCCL regime. The regimes agree on pinned traffic (stencils) and differ where collectives dominate. For MoE with $a = 0.6$, the fixed regime credits only −13.3%: holding the dense 2 GB allreduce ring in rank order makes every placement that departs from rank order pay for a scattered ring, which NCCL would in practice rebuild.

Affinity sweep. Fig. 2 varies affinity. With interleaved groups the gain grows almost linearly: −18.8%, −37.5%, −56.3%, −75.1% cross-host bytes at $a = 0.2, 0.4, 0.6, 0.8$. With groups aligned to hosts it is zero at every affinity, because block placement is already correct. The gain grows in proportion to a, so in part it reads back the generator's parameter. The rule that follows is: *placement pays when the traffic has structure that disagrees with the launcher's rank order*. Our significance test does not detect that structure reliably. MoE with $a = 0.6$ is not significant on the full graph (Table III): no null graph reaches its

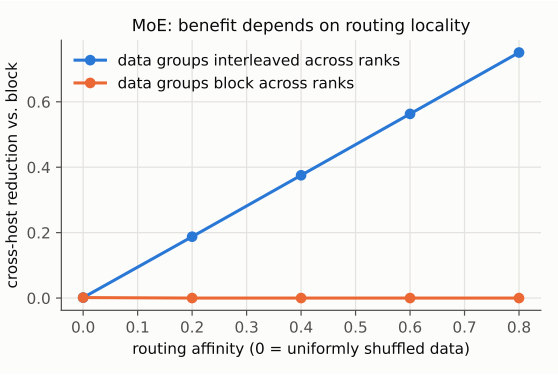


Fig. 2. E4: MoE cross-host reduction vs. block against routing affinity, NCCL regime. Scenario values.

TABLE V
ANOMALY DETECTION, HIT RATE AT RANK 1 (E5). SCENARIO VALUES.

	volume ×3	NIC at 25%	
Workload	bytes	bytes	time-weighted
MoE, $a = 0$	0.06	0.12	1.00
Megatron 8·4·2	1.00	0.12	1.00
Stencil 2D 8×8	0.97	0.12	0.75

Q ($p = 0.048$, the minimum attainable), but $z = 2.5$ falls short of 3. Yet it gains −56.3%. We suspect the dense allreduce hides the expert structure: once re-ringed it costs the same for every exact-capacity placement, yet it dominates the graph that Louvain sees. Testing significance on rank-pinned traffic alone is future work. What decides is the direct comparison: the recommended partition against block placement under the NCCL model, which is computable from a trace before anything is moved. The null-model test guards against reporting structure that is not there; it is not a gate that placement must pass.

E. Bytes are not bottlenecks (E5)

Injecting extra bytes and finding them with a bytes metric is circular, so E5 uses two anomalies (Table V). Tripling one rank's traffic is found by bytes strength in Megatron (hit rate 1.00) and the stencil (0.97), but only 0.06 of the time in skewed MoE, where hot experts already dominate strength. The real test runs one host's NIC at 25% bandwidth, which leaves bytes unchanged. Bytes strength then picks the slow host at chance (0.12; chance is 1/8). Time-weighted strength, with each edge weighted by its transfer time, finds it every time in MoE and Megatron and 0.75 of the time in the stencil. A sanity row confirms that the slowed host is the model's true bottleneck in every case. Real extraction therefore has to record per-event durations, which Chakra and the PMPI library do and NCCL INFO logs do not.

F. How much trace is needed (E6)

A 200-step MoE run ($a = 0.6$, doubled routing noise) is sampled every k-th step. Placement quality, defined as the share of the full-trace cut reduction that a placement computed from

the sample retains on the full traffic, stays at 1.000 down to 1% of steps (2 steps). Community NMI stays at 1.000, hotspot order (Kendall τ) falls only from 1.000 to 0.945, and top-10 hotspot overlap drops to 0.818 from 10% of steps onward. When routing is stationary, a few steps suffice. Non-stationary routing breaks this, and that is the subject of follow-up work.

G. Overhead (E7)

On Megatron-style graphs up to 1024 ranks (a laptop with an Intel i5-13420H), load metrics take 0.009 s, Louvain 0.060 s, METIS 0.208 s and sampled betweenness ($k = 64$ pivots) 0.252 s. The full recommendation with swap refinement takes 0.79 s at 512 ranks and 10.7 s at 1024: dense best-improvement swaps cost $O(n^2)$ per iteration and dominate. Restricting swaps to boundary vertices or updating gains incrementally is the obvious fix. Timings are machine-dependent.

VI. Discussion and Threats to Validity

What the results say. Graph analysis is most useful when it says "do nothing": for the two most common training layouts, data parallelism and Megatron in launcher order, block placement is already optimal. Its value lies in detecting mismatches: scrambled or heterogeneous launches, MPI Cartesian codes on GPU hosts, and MoE models whose routing has data-group locality. Each of these is detectable from a trace by comparing the recommended partition with rank order under the NCCL model; the significance test prevents false claims of structure but, as the MoE case shows, can miss real structure.

When a trace is not needed. If the parallelism layout is known (TP, PP, DP and EP group sizes), a scheduler can place groups by rule, for example each TP group inside one host, without any trace, and NCCL already orders its rings by topology. Trace-based analysis adds value only where structure is not visible in the configuration: data-dependent routing, irregular MPI codes, fragmented allocations, and launches whose rank order silently departs from the intended layout. Our scrambled-Megatron case is a stand-in for the last; how often it occurs in practice is untested.

Synthetic workloads. Every workload is a model. MoE routing affinity is a parameter, not an observation; load-balancing losses in real MoE training may suppress it. Megatron is modelled without sequence parallelism or a distributed optimiser.

Collective algorithms. NCCL chooses ring, tree or NVLS algorithms at run time; INFO-level logs do not say which ran, so CAGE must assume one and records the assumption. The tree is modelled as a single binary tree, which overstates per-edge bytes by up to 2× relative to NCCL's double binary tree. The NCCL regime orders rings by host but does not model NCCL's channel count or intra-host topology search.

Cost model. The bandwidth model ignores compute overlap, dependencies and congestion inside the fabric, so modelled time reductions are an upper bound on what a real job would gain from communication alone.

Extraction. The NCCL, Chakra and PMPI parsers pass tests on hand-written fixtures but have not yet been run on real logs; the PMPI library has not been compiled against a real MPI. Fortran codes may bypass a C interposition layer, depending on the MPI implementation's bindings.

Betweenness. Messages in an application-level graph do not travel through intermediate ranks, so shortest-path centrality has no physical meaning here. CAGE computes it (with distance = 1/weight) only as an experimental feature and we draw no conclusions from it.

What would confirm the results. (i) NCCL logs or Chakra traces from real multi-GPU jobs, to check that data-parallel and Megatron jobs show the predicted "no gain" and to measure MoE routing affinity directly; (ii) MPI traces of the NAS Parallel Benchmarks [33] under the PMPI library; (iii) replay of recommended placements in a simulator such as ASTRA-sim [34] or SimGrid [35], or on a cluster allocation, to replace the analytic model with measured step times.

VII. Conclusion

CAGE turns communication traces into significance-tested graphs and exact-capacity placements scored under the regime the communication library actually implements. On models of today's training and HPC workloads it reaches a deliberately modest conclusion: communication-aware placement beats the launcher default only when the traffic carries structure that disagrees with rank order. Raw modularity cannot tell structured from unstructured graphs; a null-model test can guard against false structure but can also miss real structure hidden under placement-neutral collectives, and a bytes-only graph cannot locate time bottlenecks. Real traces are the next step; the pipeline and the experiments are ready for them.

Artifact Availability

The code, the seven experiment scripts (E1–E7) and every results file behind this paper's numbers are available at https://github.com/n0tdivyansh/cage-research. `python -m pytest` runs the invariants the numbers depend on; `cage run --paper 1` regenerates all results (runtimes are machine-dependent).

References

[1] S. H. Bokhari, "On the mapping problem," *IEEE Transactions on Computers*, vol. C-30, no. 3, pp. 207–214, 1981.

[2] H. Chen, W. Chen, J. Huang, B. Robert, and H. Kuhn, "MPIPP: An automatic profile-guided parallel process placement toolset for SMP clusters and multiclusters," in *Proceedings of the 20th International Conference on Supercomputing (ICS)*, 2006, pp. 353–360.

[3] E. Jeannot, G. Mercier, and F. Tessier, "Process placement in multicore clusters: Algorithmic issues and practical techniques," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 4, pp. 993–1002, 2014.

[4] F. Pellegrini and J. Roman, "SCOTCH: A software package for static mapping by dual recursive bipartitioning of process and architecture graphs," in *High-Performance Computing and Networking (HPCN Europe)*, ser. LNCS, vol. 1067, 1996, pp. 493–498.

[5] T. Hoefler and M. Snir, "Generic topology mapping strategies for large-scale parallel architectures," in *Proceedings of the International Conference on Supercomputing (ICS)*, 2011, pp. 75–84.

[6] J. L. Träff, "Implementing the MPI process topology mechanism," in *Proceedings of the ACM/IEEE Conference on Supercomputing (SC)*, 2002.

[7] *MPI: A Message-Passing Interface Standard, Version 4.0*, Message Passing Interface Forum, 2021.

[8] R. Guimerà, M. Sales-Pardo, and L. A. N. Amaral, "Modularity from fluctuations in random graphs and complex networks," *Physical Review E*, vol. 70, p. 025101, 2004.

[9] T. Hoefler, E. Jeannot, and G. Mercier, "An overview of topology mapping algorithms and techniques in high-performance computing," in *High-Performance Computing on Complex Environments*. Wiley, 2014, pp. 73–94.

[10] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998.

[11] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *Bell System Technical Journal*, vol. 49, no. 2, pp. 291–307, 1970.

[12] A. Knüpfer *et al.*, "Score-P: A joint performance measurement run-time infrastructure for Periscope, Scalasca, TAU, and Vampir," in *Tools for High Performance Computing 2011*. Springer, 2012, pp. 79–91.

[13] S. Sridharan *et al.*, "Chakra: Advancing performance benchmarking and co-design using standardized execution traces," arXiv:2305.14516, 2023.

[14] Y. Jin, H. Wang, T. Yu, X. Tang, T. Hoefler, X. Liu, and J. Zhai, "ScalAna: Automating scaling loss detection with graph analysis," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.

[15] A. Sergeev and M. Del Balso, "Horovod: Fast and easy distributed deep learning in TensorFlow," arXiv:1802.05799, 2018.

[16] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.

[17] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, vol. 69, no. 2, pp. 117–124, 2009.

[18] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," arXiv:1909.08053, 2019.

[19] D. Narayanan *et al.*, "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[20] D. Lepikhin *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *International Conference on Learning Representations (ICLR)*, 2021.

[21] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[22] J. He, J. Zhai, T. Antunes, H. Wang, F. Luo, S. Shi, and Q. Li, "FasterMoE: Modeling and optimizing training of large-scale dynamic pre-trained models," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2022, pp. 120–134.

[23] Y. Peng, Y. Zhu, Y. Chen, Y. Bao, B. Yi, C. Lan, C. Wu, and C. Guo, "A generic communication scheduler for distributed DNN training acceleration," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 16–29.

[24] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, "TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.

[25] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," *Physical Review E*, vol. 69, p. 026113, 2004.

[26] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, p. P10008, 2008.

[27] S. Fortunato and M. Barthélemy, "Resolution limit in community detection," *Proceedings of the National Academy of Sciences*, vol. 104, no. 1, pp. 36–41, 2007.

[28] S. Maslov and K. Sneppen, "Specificity and stability in topology of protein networks," *Science*, vol. 296, no. 5569, pp. 910–913, 2002.

[29] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955.

[30] P. W. Holland, K. B. Laskey, and S. Leinhardt, "Stochastic blockmodels: First steps," *Social Networks*, vol. 5, no. 2, pp. 109–137, 1983.

[31] L. Danon, A. Díaz-Guilera, J. Duch, and A. Arenas, "Comparing community structure identification," *Journal of Statistical Mechanics: Theory and Experiment*, p. P09008, 2005.

[32] L. Hubert and P. Arabie, "Comparing partitions," *Journal of Classification*, vol. 2, pp. 193–218, 1985.

[33] D. H. Bailey *et al.*, "The NAS parallel benchmarks," *International Journal of Supercomputer Applications*, vol. 5, no. 3, pp. 63–73, 1991.

[34] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-SIM: Enabling SW/HW co-design exploration for distributed DL training platforms," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020.

[35] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, "Versatile, scalable, and accurate simulation of distributed applications and platforms," *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014.